Pattern-Use Case Mapping Matrix

Document section author: Alejandro Blas Piris (261118081)
Note to team: Carlos, please integrate this section into the main Software Requirements Specification (SRS) document under the Architecture and Patterns section.

The following matrix defines the specific software design patterns selected for the platform's core use cases. Each pattern is assigned to solve a specific architectural challenge, maximizing the flexibility, modularity, and maintainability of the Business Layer in our Layered Architecture.

ID	Use Case	Assigned Pattern	Application and Architectural Justification
UC1	Register & Create Profile	Factory Method + Adapter	Factory Method is applied in the 'UserService' to centralize the creation logic for different user profiles (Erasmus vs. Local students). This ensures modularity, as new profile types can be added without modifying existing instantiation logic. Adapter is used to wrap external services, specifically the eduroam authentication mechanism. It isolates our Business Layer from external API changes, enhancing maintainability by allowing us to simulate the service with a mock during development.
UC2	Find a Match	Strategy	The Strategy pattern provides runtime flexibility to the 'MatchingService'. Depending on user preferences, the algorithm used to find compatible students needs to change (e.g., matching by language exchange goals, by hobbies, or by sports). Strategy allows these different matching criteria to be encapsulated into interchangeable classes, meaning new filters can be easily introduced without altering the core matching service.

ID	Use Case	Assigned Pattern	Application and Architectural Justification
UC3	Create/Join Event & Get Notified	Observer	The Observer pattern perfectly fits the notification requirement. The 'EventService' acts as the Subject, while subscribed users act as Observers. When an event is updated or a new match is found, the system automatically notifies all interested users. This heavily decouples event management logic from notification delivery, increasing the overall modularity of the system.
UC4	Report Inappropriate Content	Chain of Responsibility	Chain of Responsibility organizes the moderation workflow into a sequence of handlers (Automatic Filter → Moderator → System Admin). When a report is submitted via 'ReportService', it passes along this chain until a handler can resolve it. This provides extreme maintainability, as escalation rules or new moderation levels can be added without rewriting the reporting service structure.